

Unsupervised Learning

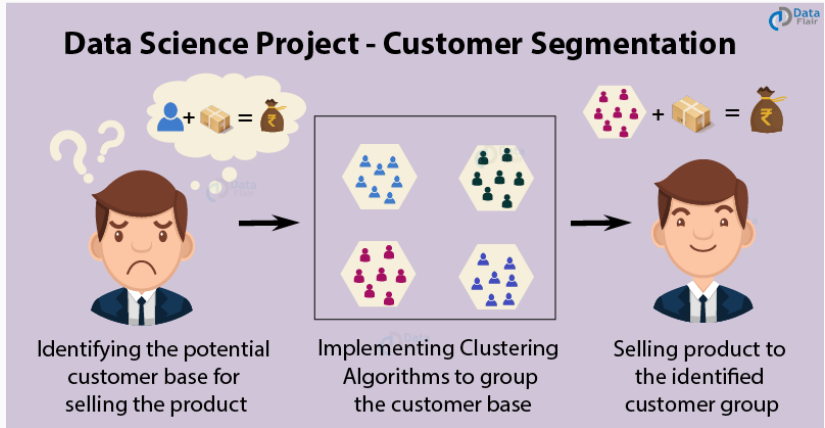
Clustering

Mahdi Shafiee Kamalabad & Maarten Cruyff

What is Clustering?

- Clustering is an **unsupervised learning** technique that aims to **group observations into clusters** such that:
 - observations **within the same cluster are similar**
 - observations **in different clusters are dissimilar**
- Similarity is usually defined using a **distance or dissimilarity measure** (e.g. Euclidean distance).

Customer segmentation (with k-means)



Centroid-based Clustering

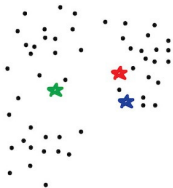
Centroid-based Clustering

- Centroid-based clustering groups data points around a **center point** called a **centroid**.
- It creates a **flat (non-hierarchical) partition** of the data into a **fixed number of clusters**.
- Each cluster is represented by a **centroid**, usually the mean/average position of the points in that cluster.
- In contrast to hierarchical clustering, these methods do **not** create a tree or nested groups.
- The most widely used centroid-based clustering algorithm is **K-means**.
- Centroid-based methods are fast and useful, but they can be **sensitive to the initial choice of centroids and to outliers**.

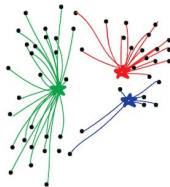
Centroid-based Clustering: K-means Clustering

PUT KEBAB KIOSKS IN THE OPTIMAL WAY

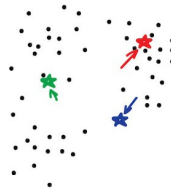
(also illustrating the K-means method)



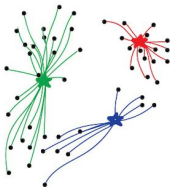
1. Put kebab kiosks in random places in city



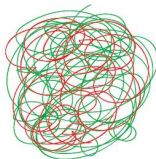
2. Watch how buyers choose the nearest one



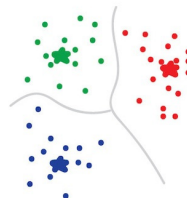
3. Move kiosks closer to the centers of their popularity



4. Watch and move again



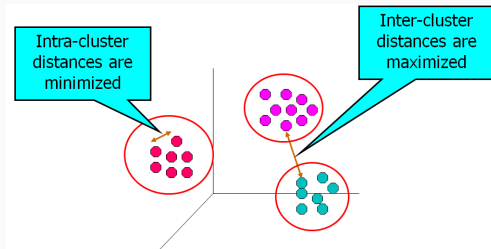
5. Repeat a million times



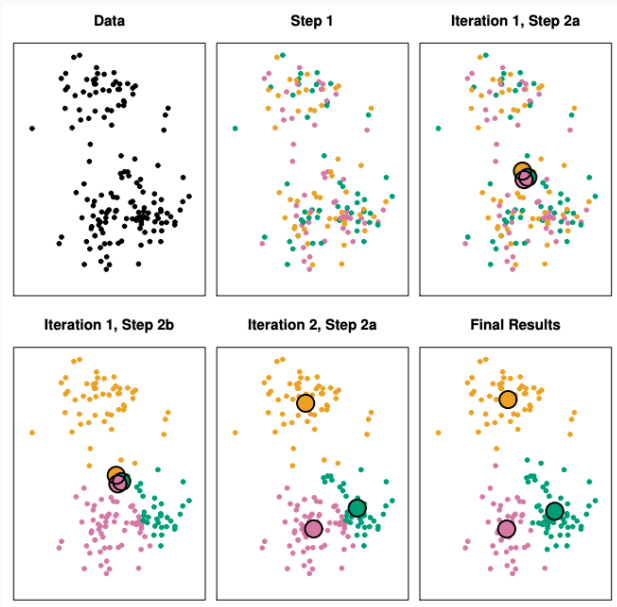
6. Done!

K-means Algorithm

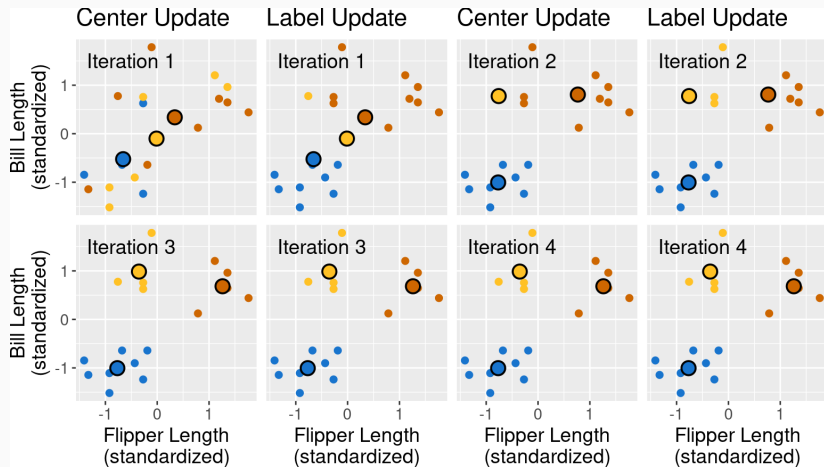
1. Specify the number of clusters K
2. Initialize K cluster centroids (often randomly)
3. Assign each observation to the **nearest centroid**
4. Update each centroid as the **mean of its assigned observations**
5. Repeat steps 3 and 4 until the assignments no longer change



K-means clustering



Steps: Penguins dataset



Source: Data Science Book - Clustering

K-means: Common Issues and Solutions 1

Choosing the number of clusters K

- In K-means, we must choose the number of clusters K before running the algorithm.
- **Elbow method:** try different values of K , then look for the point where improvement starts to slow down.
- **Silhouette method:** checks whether each point fits well in its own cluster and is far from other clusters.
- **Gap statistic:** compares our clustering result with clustering on random data.
- See “here” for a comparison of methods.

K-means: Common Issues and Solutions 2

Local optima

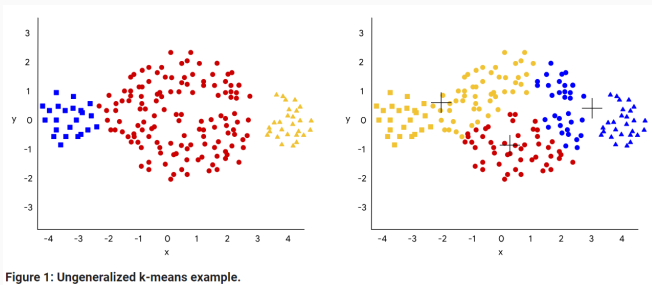
- K-means does not always find the best possible clustering.
- Different random starting centroids can give different final results.
- Use **multiple random starts** using `nstart` to get a more stable result.

Sensitivity to scale

- K-means uses **Euclidean distance**, so variables with larger scales can dominate the clustering.
- Features should usually be **standardized** before applying K-means.
- Clustering on **principal components** can sometimes improve results, especially when variables are correlated or noisy.

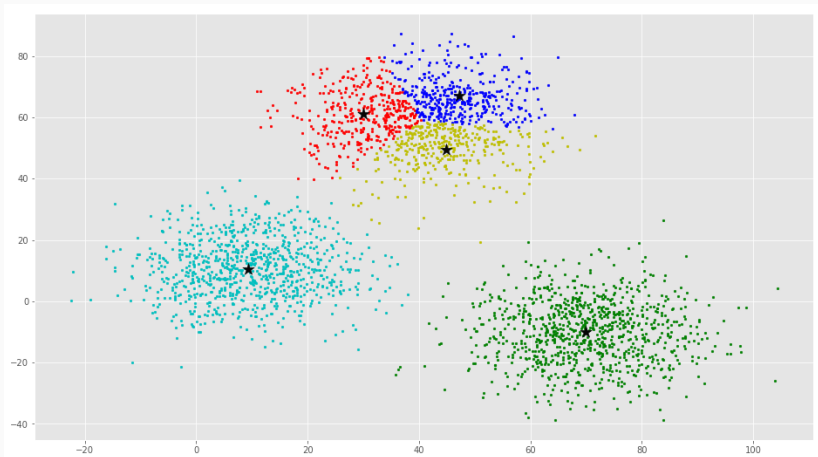
Local optima in K-means

- Different initializations can lead to **different clustering results**, even on the same data.



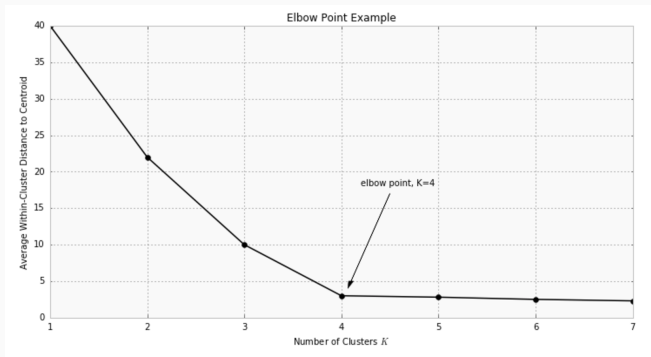
Choosing the Number of Clusters K

Would $K = 3$ be a better solution than the current choice?



Elbow Method

- The elbow method chooses K by inspecting how the **within-cluster sum of squares (WSS)** decreases as K increases.
- Look for the point where adding more clusters gives only a small improvement.



Example: The iris dataset

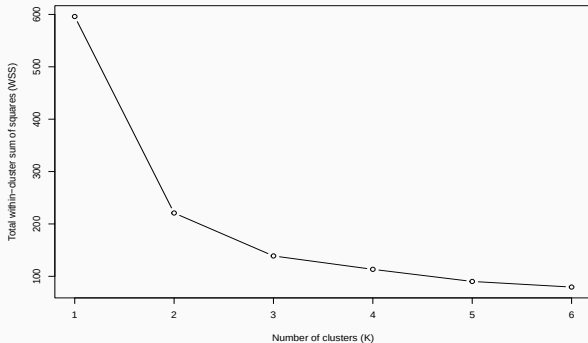
- contains measurements of **150 flowers** from three species:
 - *Setosa*, *Versicolor*, *Virginica*
- For each flower, **four numerical features** are recorded:
 - sepal length, sepal width
 - petal length, petal width

K-means on the Iris Data: Elbow Method (WSS)

- Apply K-means to the **scaled** Iris features
- Fit models for $K = 1, \dots, 6$ using **20 random starts**
- Plot the total within-cluster sum of squares (WSS) versus K

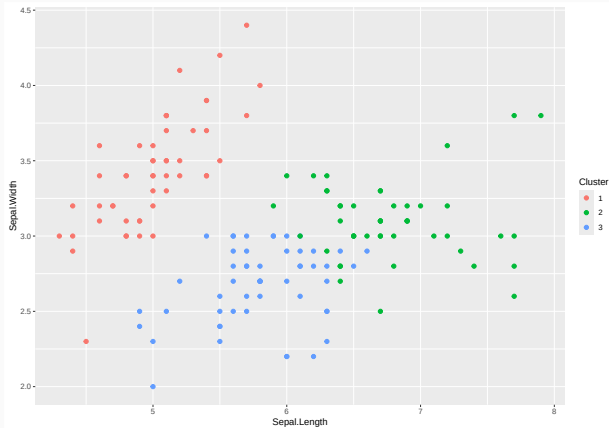
K-means on the Iris Data: Elbow Method (WSS)

```
X <- scale(iris[, 1:4])  
wss <- sapply(1:6, function(k) {  
  kmeans(X, centers = k, nstart = 25)$tot.withinss  
})  
plot(1:6, wss, type = "b", xlab = "Number of clusters (K)",  
     ylab = "Total within-cluster sum of squares (WSS)")
```



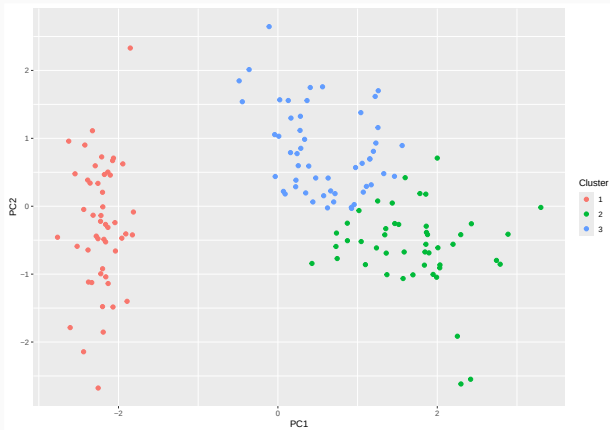
Example: K-means Clustering on iris data

- In this example, the clustering happens in 4 dimensions, which is difficult to visualize directly.
- The plot below shows only a 2-dimensional visualization of the clusters. Is that enough?



Visualizing K-means Clusters in PCA Space

- PCA is used here to visualize the K-means clusters in two dimensions, while the clustering itself was performed on the original scaled variables.



km result

K-means in R

```
# Standardize features before K-means
x_scaled <- scale(x)

# Run K-means with K clusters
k_fit <- kmeans(x_scaled, centers = K, nstart = 25)
# x_scaled: standardized data matrix
# centers = K: number of clusters to create
# nstart = 25: run K-means 25 times with different starting centroids
# and keep the best result

# Main outputs from the K-means result
print(k_fit)           # summary: cluster sizes, centers, WSS, etc.
k_fit$cluster          # cluster assignment for each observation
k_fit$centers          # final cluster centroids
k_fit$tot.withinss     # total within-cluster sum of squares
k_fit$size             # number of observations in each cluster
```

Connectivity-based Clustering

Connectivity-based Clustering

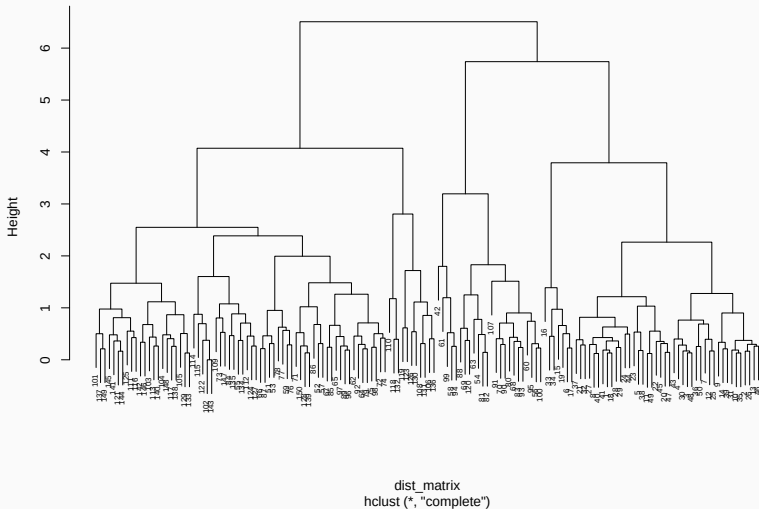
- Also known as **hierarchical clustering**.
- Groups data points based on how **close** or **connected** they are to each other.
- Organizes clusters into a **hierarchy**, or levels.
- The result is a **tree of clusters**, called a **dendrogram**.
- Unlike K-means, we do **not** need to choose the number of clusters before building the hierarchy.
- To get a final set of clusters, we can **cut the dendrogram** at a chosen level.

Dendrogram: Visualizing the Hierarchy

- A **dendrogram** is a tree diagram used to show hierarchical clustering.
- It shows which data points or clusters are joined together step by step.
- The height of each join shows how different the joined groups are.
- By choosing a **cut level**, we can decide how many final clusters we want.

Example: Hierarchical Clustering

Hierarchical Clustering Dendrogram



Hierarchical clustering methods can be divided into two main types:

1. **Agglomerative (bottom-up)**
2. **Divisive (top-down)**

Agglomerative Clustering (Bottom-up)

Agglomerative clustering starts with **each observation as its own cluster** and **iteratively merges** the most similar clusters until a stopping criterion is reached.

Algorithm (Agglomerative / Bottom-up, Greedy)

0. **Standardize the features** (recommended for distance-based clustering)
1. Start with **each observation as its own cluster**
2. Compute the **pairwise distances** between all clusters
3. **Merge the two closest clusters** according to a chosen **linkage criterion** (e.g. single, complete, average)
4. Repeat steps 2-3 until **all observations are merged into one cluster**
5. Visualize the result using a **dendrogram** (and optionally cut the tree to obtain a desired number of clusters)

Agglomerative Clustering (Bottom-up): Step 1

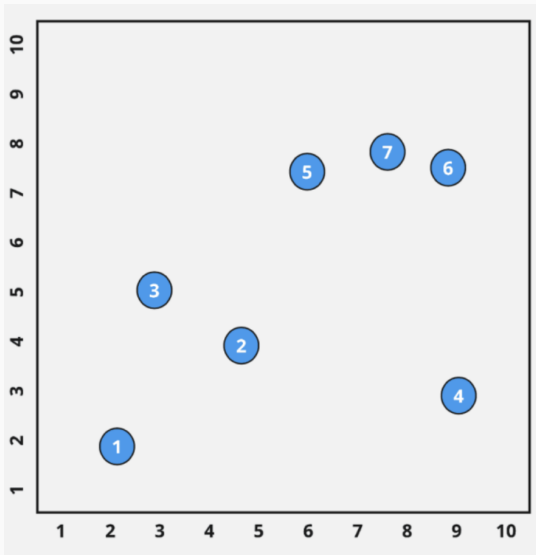


Figure 1: Agglomerative clustering: step 1

Agglomerative Clustering (Bottom-up): Step 2

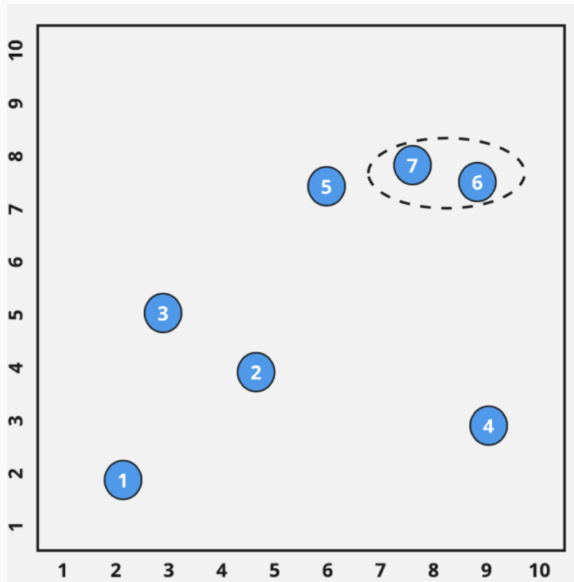


Figure 2: Agglomerative clustering: step 2

Agglomerative Clustering (Bottom-up): Step 3

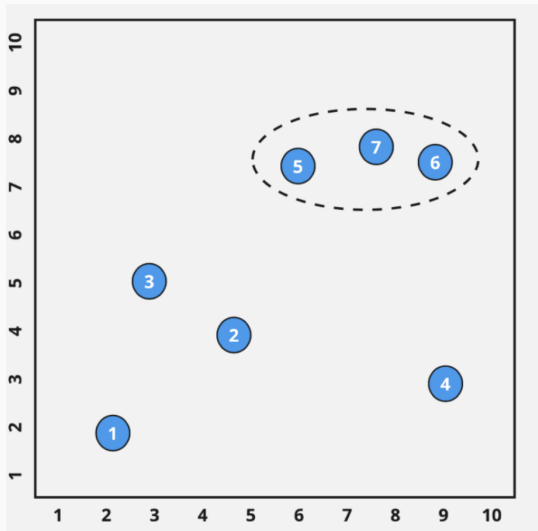


Figure 3: Agglomerative clustering: step 3

Agglomerative Clustering (Bottom-up): Step 4 and step 5

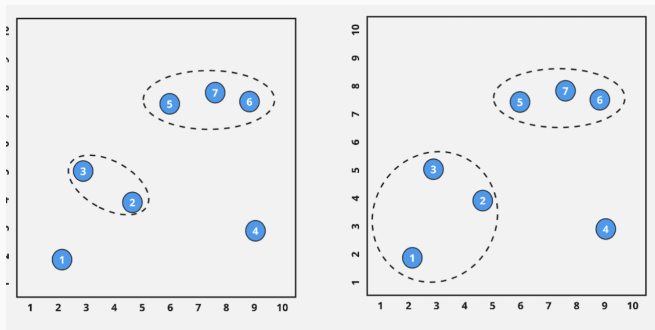


Figure 4: Agglomerative clustering: step 4 and step 5

Agglomerative Clustering (Bottom-up): Step 6 and step 7

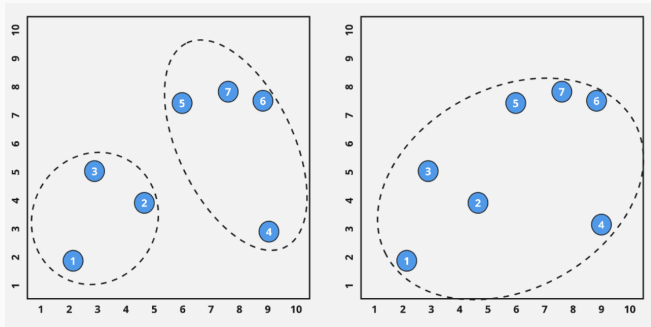


Figure 5: Agglomerative clustering: step 6 and step 7

- Once all the clusters are combined into a big cluster. We develop the Dendrogram to divide the clusters.

Divisive Clustering (Top-down)

Divisive clustering starts with **all observations in a single cluster** and **iteratively splits** clusters into smaller ones.

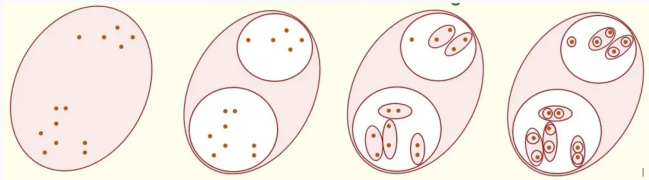
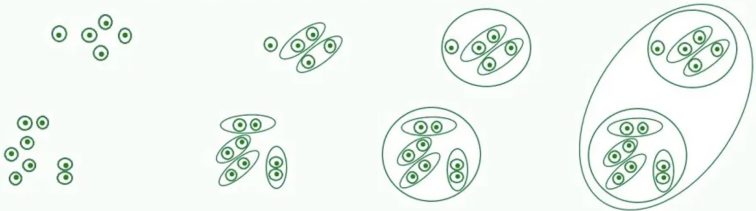


Figure 6: Divisive Clustering (Top-down)

Agglomerative Clustering and Divisive Clustering

Agglomerative Hierarchical Clustering



Divisive Hierarchical Clustering

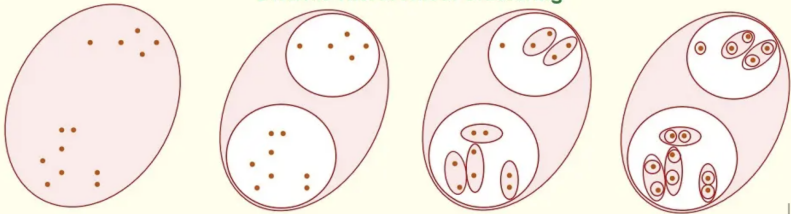


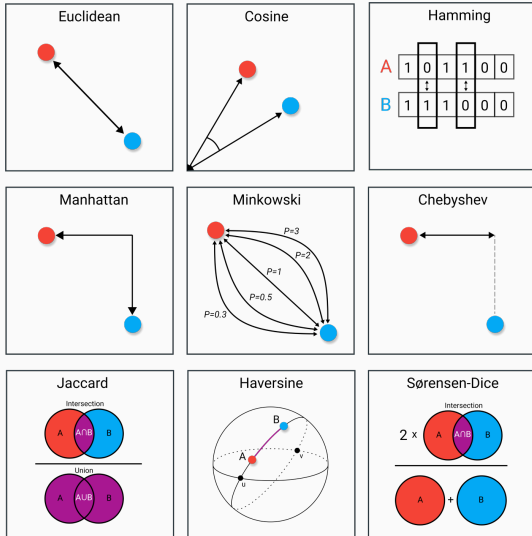
Figure 7: Agglomerative Clustering and Divisive Clustering

Compute Distances

Hierarchical clustering starts by calculating the **distance** between every pair of observations.

- A **distance measure** tells us how different two observations are.
- Small distance means the observations are **similar**.
- Large distance means the observations are **different**.
- Common distance measures include:
 - **Euclidean distance**: straight-line distance
 - **Manhattan distance**: distance by moving along grid lines
 - **Cosine distance**: compares the direction/pattern of observations
- The choice of distance measure can change the final clustering result.

Complete list of distance measures



Source: Data Science - Must-Know Distance Metrics

Which Distance Measure?

The choice of distance measure is crucial in clustering.

- **Depends on the data type**
 - continuous, binary, categorical, text, etc.
- **Often requires empirical evaluation**
 - try multiple distance measures and compare results (trial and error)
- **Be cautious with Euclidean distance in high dimensions**
 - in high-dimensional spaces, distances tend to become less informative
 - many points end up at similar distances from each other



350



+50



A great summary of non-intuitive results in higher dimensions comes from "[A Few Useful Things to Know about Machine Learning](#)" by Pedro Domingos at the University of Washington:

[O]ur intuitions, which come from a three-dimensional world, often do not apply in high-dimensional ones. In high dimensions, most of the mass of a multivariate Gaussian distribution is not near the mean, but in an increasingly distant "shell" around it; and most of the volume of a high-dimensional orange is in the skin, not the pulp. If a constant number of examples is distributed uniformly in a high-dimensional hypercube, beyond some dimensionality most examples are closer to a face of the hypercube than to their nearest neighbor. And if we approximate a hypersphere by inscribing it in a hypercube, in high dimensions almost all the volume of the hypercube is outside the hypersphere. This is bad news for machine learning, where shapes of one type are often approximated by shapes of another.

The article is also full of many additional pearls of wisdom for machine learning.

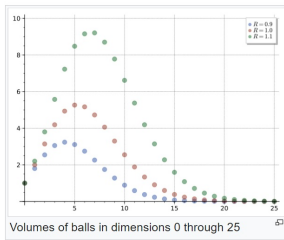


Figure 8: <https://stats.stackexchange.com/a/99191/> and https://en.wikipedia.org/wiki/Volume_of_an_n-ball

Distance Function in R: dist()

The function `dist()` computes **pairwise distances** between observations.

```
# Use the first 3 Iris observations and scale the numeric variables  
x <- scale(iris[1:3, -5])
```

```
# Euclidean distance: straight-line distance  
dist(x, method = "euclidean")
```

```
##           1           2  
## 2 2.224268  
## 3 2.901905 2.152110
```

```
# Manhattan distance: city-block distance  
dist(x, method = "manhattan")
```

```
##           1           2  
## 2 2.986799  
## 3 4.924130 3.526770
```

```
# Maximum distance: largest single-variable difference  
dist(x, method = "maximum")
```

```
##           1           2  
## 2 1.986799  
## 3 2.000000 1.732051
```

Linkage Methods

How do we measure the distance between two clusters?

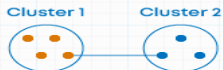
- After computing distances between individual observations, we need a **linkage method** to define the distance between groups of observations.
- **Single linkage**
 - Uses the distance between the **closest two points** in the clusters.
- **Complete linkage**
 - Uses the distance between the **farthest two points** in the clusters.
- **Average linkage**
 - Uses the **average distance** between all pairs of points in the two clusters.
- **Centroid linkage**
 - Uses the distance between the **centers**, or centroids, of the clusters.
- **Ward's method**
 - Merges the clusters that cause the **smallest increase in within-cluster variation**.

Linkage Methods

- **Single Linkage**

$$D(c_1, c_2) = \min D(x_1, x_2)$$

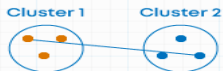
Minimum distance or distance between closest elements in clusters



- **Complete Linkage**

$$D(c_1, c_2) = \max D(x_1, x_2)$$

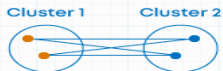
Maximum distance between elements in clusters



- **Average Linkage**

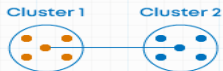
$$D(c_1, c_2) = \frac{1}{|c_1|} \frac{1}{|c_2|} \sum \sum D(x_1, x_2)$$

Average of the distances of all pairs



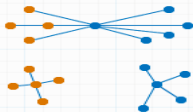
- **Centroid Method**

Combining clusters with minimum distance between the centroids of the two clusters



- **Ward's Method**

- Combining clusters where increase in within cluster variance is to the smallest degree.
- Objective is to minimize the total within cluster variance



Compute hierarchical clustering in R : `hclust()`

```
# Step 1: compute pairwise distances
d <- dist(scale(<data>), method = "euclidean")

# Step 2: apply hierarchical clustering
hc <- hclust(d, method = "complete")

# Other linkage options:
# method = "single"
# method = "average"
# method = "centroid"
# method = "ward.D2"
```

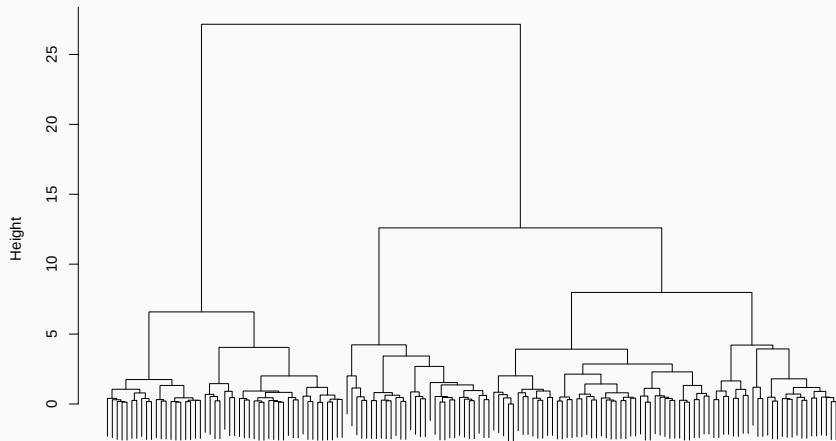
- Plot the dendrogram

```
# Plot the dendrogram
plot(hc, labels = <observation_names>) # default labels are row names

# Add rectangles around k clusters
rect.hclust(hc, k = 3) # plot rectangles
```

Example in R: iris data

Hierarchical Clustering of Iris Data



dist_matrix
hclust (*, "ward.D2")

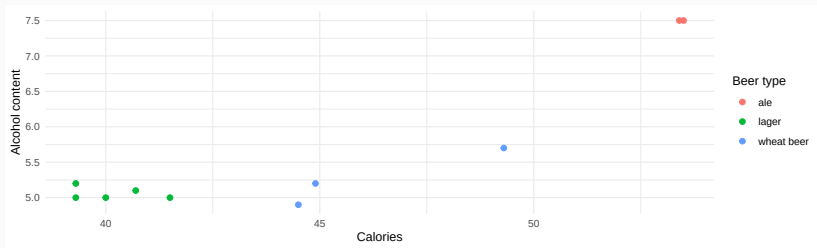
Beer Example

We cluster **10 types of beer** using only two measurements:

- **Calories**
- **Alcohol content**

The true beer types are:

- lager
- wheat beer
- ale



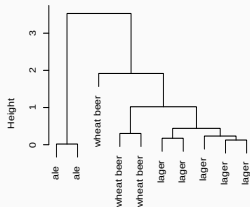
Question: Can hierarchical clustering recover the beer types using only calories and alcohol content?

Data

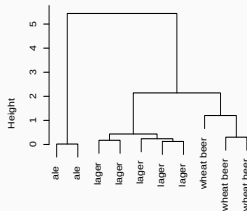
##		type	calories	alcohol
## 1		lager	39.3	5.2
## 2		lager	39.3	5.0
## 3		lager	40.0	5.0
## 4		lager	40.7	5.1
## 5		lager	41.5	5.0
## 6	wheat	beer	44.5	4.9
## 7	wheat	beer	44.9	5.2
## 8	wheat	beer	49.3	5.7
## 9		ale	53.5	7.5
## 10		ale	53.4	7.5

Comparison of Linkage Methods (Beer Data)

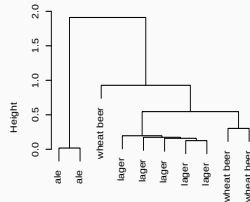
Complete linkage



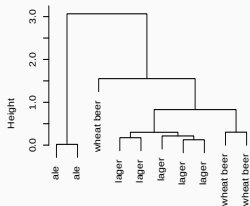
Ward.D2



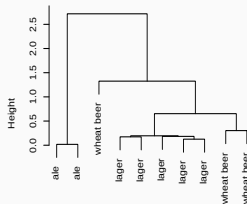
Single linkage



Average linkage



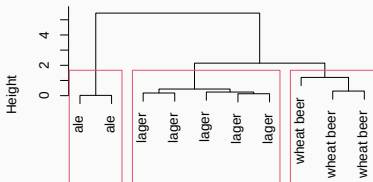
Centroid linkage



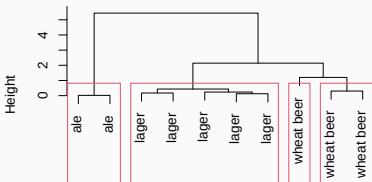
Choosing the Number of Clusters (Beer Data)

After hierarchical clustering, we choose the final number of clusters by **cutting the dendrogram** into k groups.

Ward.D2: $k = 3$



Ward.D2: $k = 4$



- $k = 3$ cuts the dendrogram into 3 groups, which may correspond to the three known beer types: lager, wheat beer, and ale.
- $k = 4$ cuts the same dendrogram into 4 groups, so one of the previous groups is split into two smaller subgroups.
- There is no universally “correct” value of k ; the best choice depends on the goal of the analysis.

Comparing K-means vs Hierarchical Clustering

- **K-means clustering**

- Fast and computationally efficient.
- Requires specifying the number of clusters K in advance.
- Sensitive to initialization and outliers.
- Works best when clusters are compact, roughly spherical, and of similar size.

- **Hierarchical clustering**

- Produces a hierarchy of clusters, visualized with a dendrogram.
- Does not require specifying the number of clusters before building the hierarchy.
- Useful for exploring relationships between observations and clusters.
- Can be computationally expensive for large datasets.

Summary So Far

So far, we have covered two common clustering approaches:

- **Centroid-based clustering**
 - Example: **K-means**
- **Connectivity-based clustering**
 - Example: **Hierarchical clustering**

Clustering methods also include other approaches:

- **Distribution-based clustering**
 - Example: **Gaussian Mixture Models (GMMs)**
 - Assumes the data are generated from a mixture of probability distributions.
 - See here: [Gaussian Mixture Model clustering in R](#)
- **Density-based clustering**
 - Example: **DBSCAN**
 - Finds clusters as dense regions of points separated by sparse regions.
 - See here: [DBSCAN clustering in machine learning](#)

- Clustering is a powerful tool for **exploratory data analysis**, with many different algorithms available.
- There is **no single best clustering method**; the choice depends on the data characteristics, the problem context, and the analysis goal.
- We will apply these ideas in the lab 4B.